1. Physical Model of the Source System

The source system is a Bike Store OLTP database consisting of 8 interrelated tables that capture sales, inventory, customers, staff, products, and store operations.

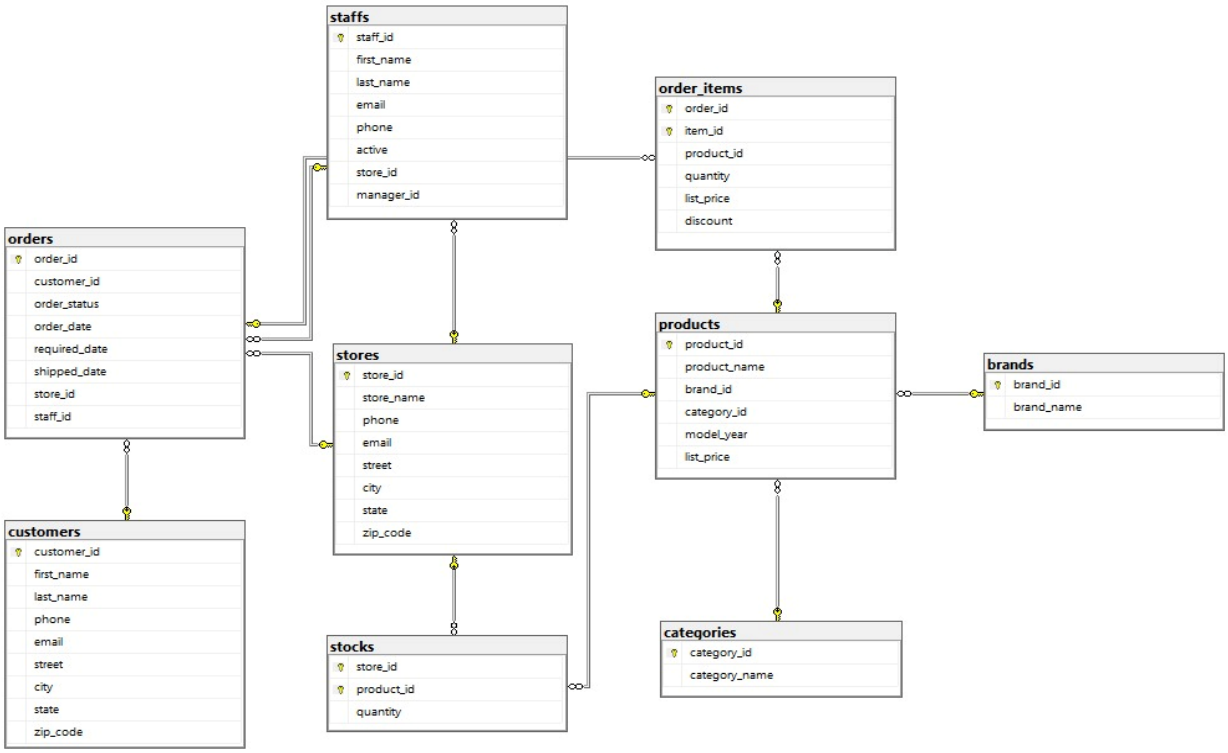


Table	Description	Key Columns
customers	Stores customer personal and contact information	customer_id (PK), first_name, last_name, email, city, state
stores	Contains store location and contact details	store_id (PK), store_name, city, state, zip_code
staffs	Employee records with store and manager relationships	staff_id (PK), store_id (FK), manager_id (FK)
products	Product catalog with brand and category references	product_id (PK), brand_id (FK), category_id (FK), list_price
brands	Lookup table for product brands	brand_id (PK), brand_name
categories	Lookup table for product categories	category_id (PK), category_name
orders	Customer orders with status and shipping details	order_id (PK), customer_id (FK), store_id (FK), staff_id (FK)
order_items	Line items for each order including price and discount	order_id (FK), item_id, product_id (FK), quantity, discount
stocks	Current inventory levels per product per store	store_id (FK), product_id (FK), quantity

The source database was loaded into SQL Server (BikeStoreOLTP) from CSV files using BULK INSERT, with referential integrity enforced via foreign key constraints.

2. Dimensional Model

a. Business Processes

Three core business processes were identified for modeling:

- Sales Analysis — tracking individual product sales per order item
- Order Management — monitoring order-level performance, statuses, and customer behavior
- Inventory Management — analyzing current stock levels per product per store

b. Grain of Each Fact Table

Fact Table	Grain Description
Fact_Sales	Each record represents one line item (a single product) within a single order — the finest level of sales detail.
Fact_Orders	Each record represents one complete order placed by a customer, summarizing the total items and revenue per order.
Fact_Inventory	Each record represents the current stock quantity of one product in one store at the time the ETL was run.

c. Type of Each Fact Table

Fact Table	Type	Reason
Fact_Sales	Transactional	Records individual sales transactions — each row is a distinct event (item sold).
Fact_Orders	Transactional	Records individual order events — each row is a distinct order placed.
Fact_Inventory	Periodic Snapshot	Captures the inventory state at a specific point in time (ETL run date).

d. Dimensions and Their Types

Dimension	Type	Description
Dim_Date	SCD Type 1 - Conformed	Shared across all 3 Facts. Contains date attributes: year, quarter, month, day, day_of_week, is_weekend.
Dim_Customer	SCD Type 2	Tracks historical changes to customer full_name and city. Includes valid_from, valid_to, is_current columns.

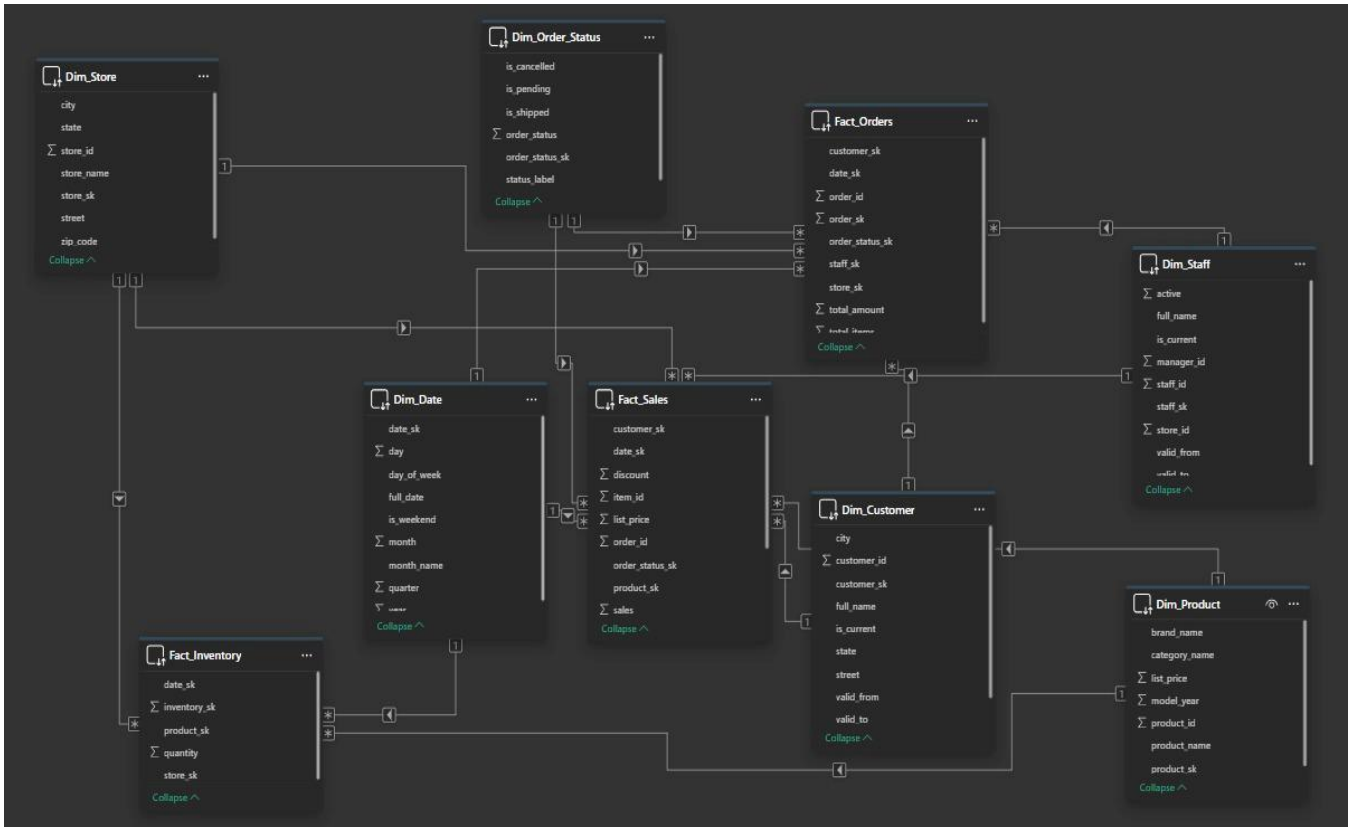
Dimension	Type	Description
Dim_Product	SCD Type 1 — Conformed	Shared across Fact_Sales and Fact_Inventory. Contains denormalized brand_name and category_name.
Dim_Store	SCD Type 1 — Conformed	Shared across all 3 Facts. Contains store location details.
Dim_Staff	SCD Type 2	Tracks historical changes to staff full_name and store assignment.
Dim_Order_Status	Junk Dimension (SCD Type 1)	Combines low-cardinality order status flags: status_label, is_shipped, is_cancelled, is_pending. Contains only 4 rows.

e. Measures in Fact Tables

Fact Table	Measure	Type	Description
Fact_Sales	quantity	Additive	Number of units sold for a specific product in the order item
Fact_Sales	list_price	Non-Additive	Unit price of the product at time of sale
Fact_Sales	discount	Non-Additive	Discount rate applied to the order item
Fact_Sales	total_amount	Additive	Calculated as quantity × list_price × (1 - discount)
Fact_Orders	total_items	Additive	Total number of distinct items in the order
Fact_Orders	total_amount	Additive	Total revenue generated by the order
Fact_Inventory	quantity	Semi-Additive	Current stock quantity — can be summed across stores but not across time

f. Physical Model (Galaxy Schema):

The final dimensional model follows a Galaxy Schema (multiple fact tables sharing conformed dimensions). The schema includes 3 Fact Tables and 6 Dimensions with the following key design



decisions:

- Dim_Date, Dim_Store, and Dim_Product are Conformed Dimensions shared across multiple Fact Tables.
- Dim_Customer and Dim_Staff implement SCD Type 2 to preserve historical changes.
- Dim_Order_Status is a Junk Dimension combining low-cardinality order status flags.
- order_id and item_id in Fact_Sales are Degenerate Dimensions — stored directly in the Fact Table without a separate dimension.
- order_id in Fact_Orders is also a Degenerate Dimension.

The schema was implemented in SQL Server (BikeStoreDWH) with full referential integrity enforced via foreign key constraints between all Fact Tables and their related Dimensions.

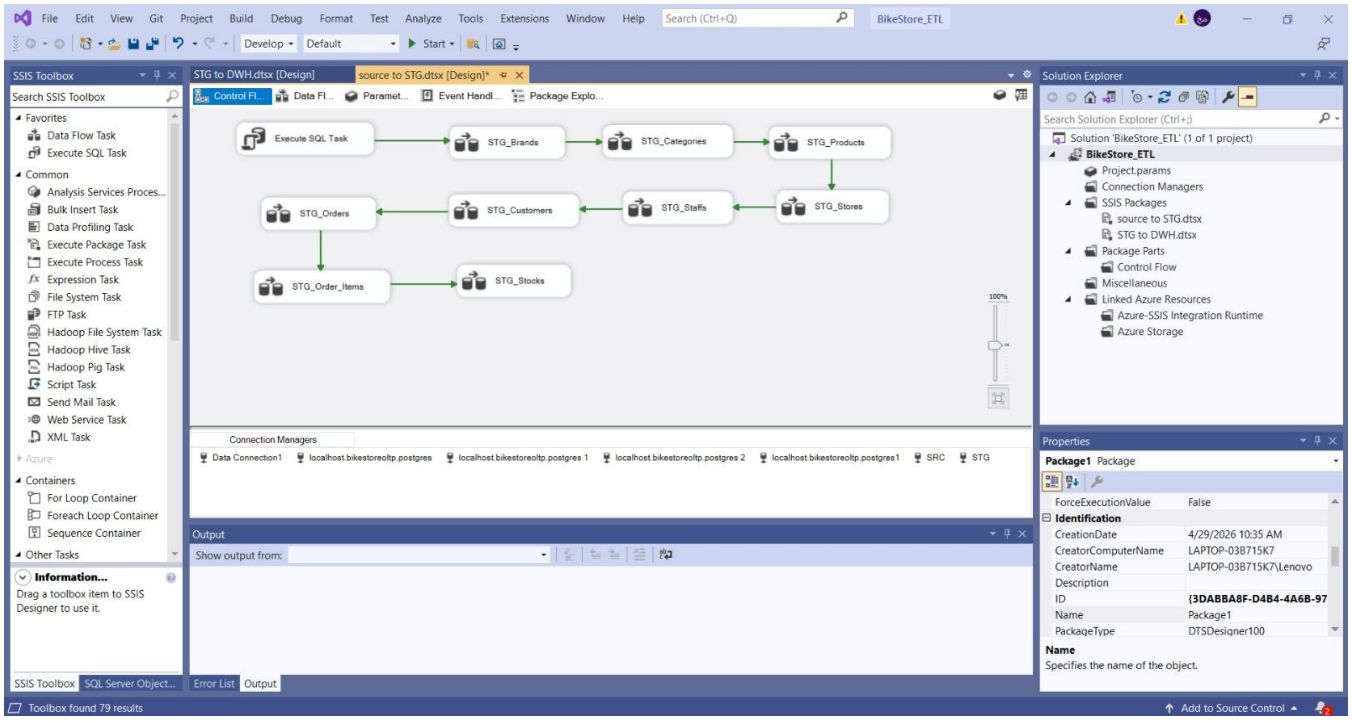
3. SSIS Data Flow and Control Flow Tasks

The ETL process was implemented using two SSIS packages:

- PKG_01 — Source to STG: Extracts data from the OLTP source (BikeStoreOLTP) and loads it into the Staging Area (BikeStoreSTG).

- PKG_02 — STG to DWH: Transforms and loads data from the Staging Area into the Data Warehouse (BikeStoreDWH).

Control Flow — Source to STG

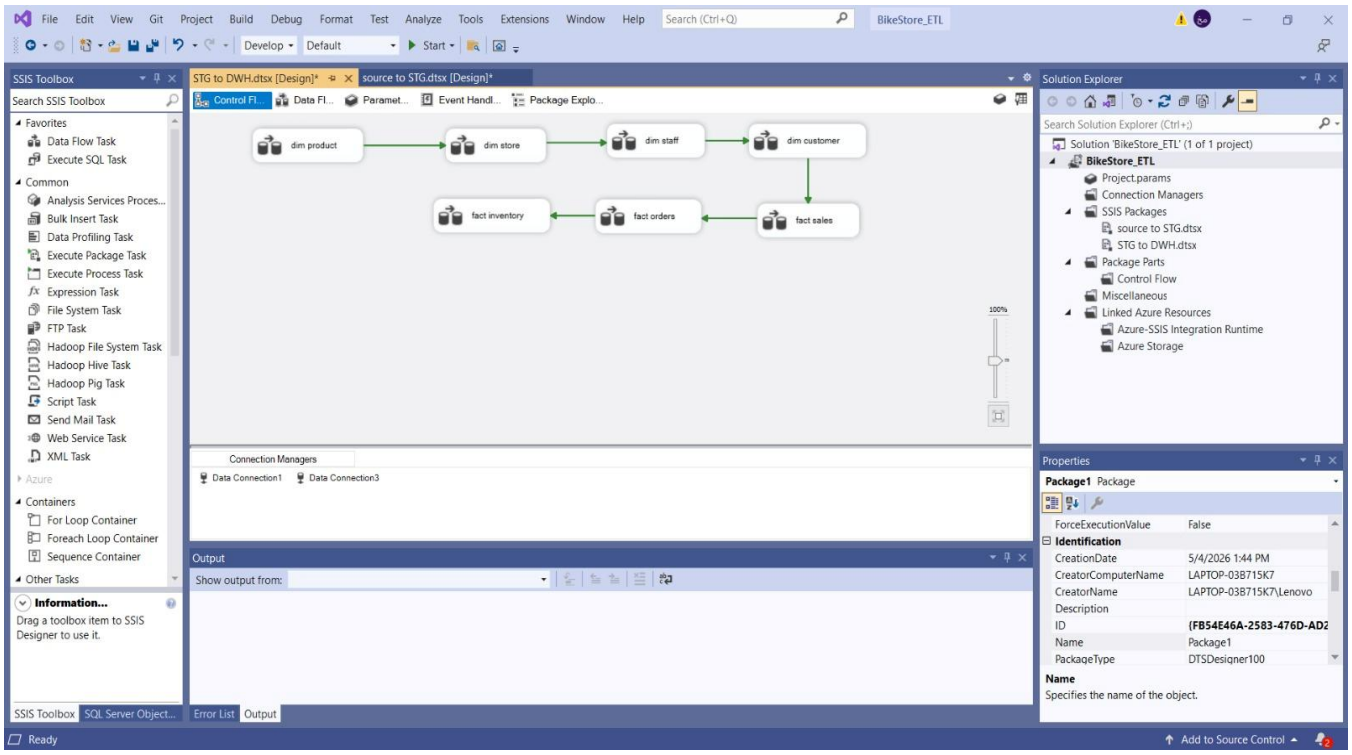


This **Control Flow** extracts raw data from the **BikeStore OLTP source system** and loads it into the **Staging layer**.

It starts with an **Execute SQL Task** to prepare the staging tables, then executes the following Data Flow Tasks sequentially:

- **STG_Brands → STG_Categories → STG_Products → STG_Stores → STG_Staffs → STG_Customers → STG_Orders → STG_Order_Items → STG_Stocks**

Control Flow — STG to DWH



This SSIS package handles the **Control Flow** of the ETL pipeline, extracting data from the source system and loading it into the **Staging (STG)** layer.

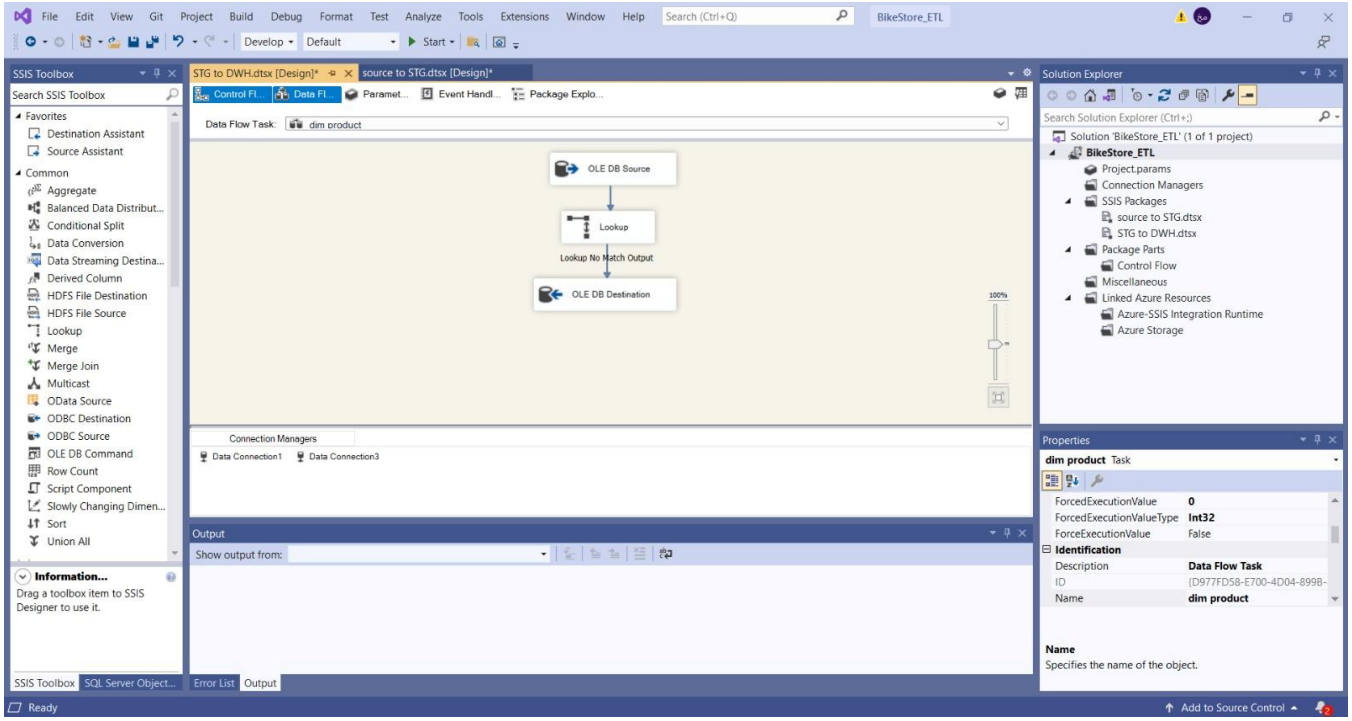
It runs multiple **Data Flow Tasks** in sequence:

- **Dimensions first:** dim_product, dim_store, dim_staff, dim_customer
- **Facts after:** fact_inventory, fact_orders, fact_sales

Two **Connection Managers** manage the connection between the source and staging database, ensuring data is landed cleanly before moving to the next phase of the pipeline.

Data Flow Task Details

Dim Product



This **Data Flow Task** handles the loading of the **Dim_Product** dimension table from the Staging layer into the Data Warehouse.

The flow consists of 3 components:

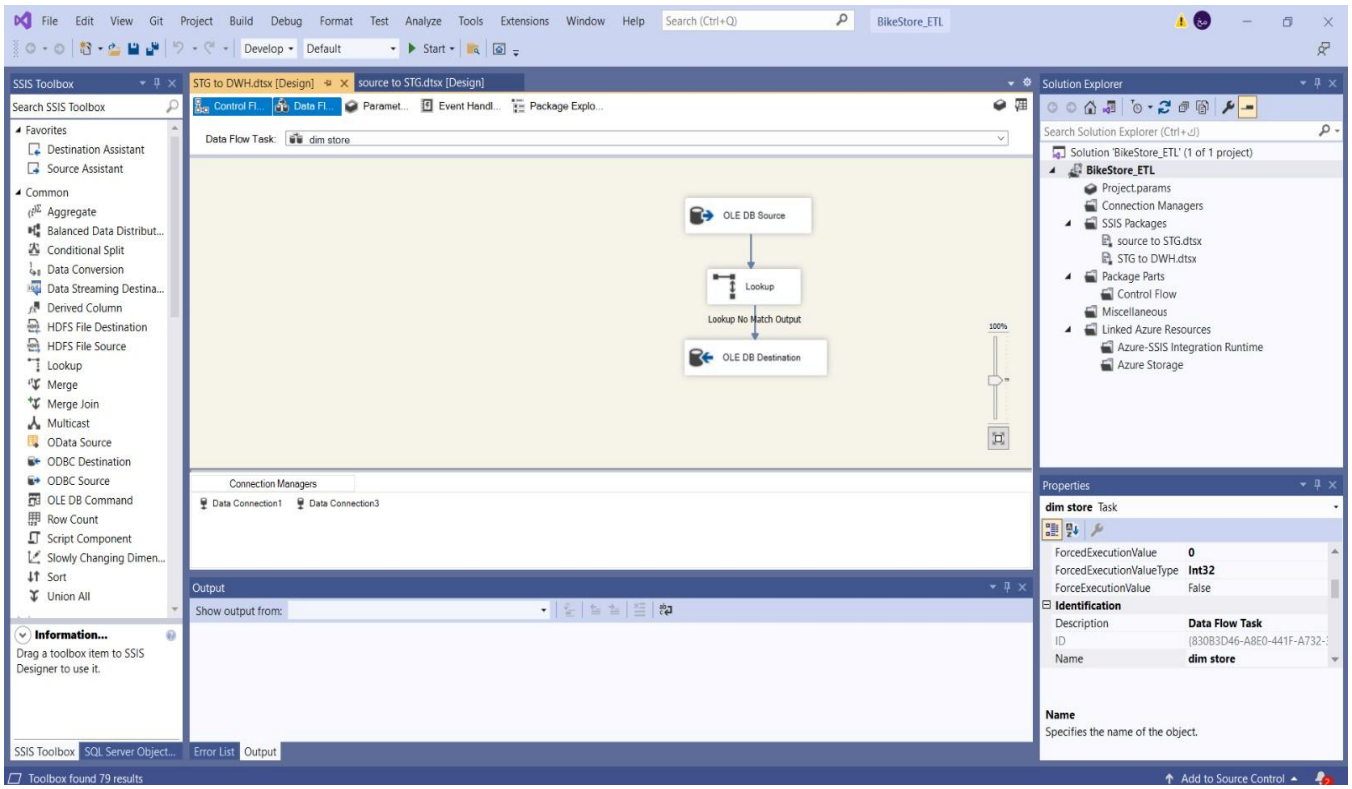
1. OLE DB Source Reads the product data from the staging table **STG_Products** (joined with STG_Brands and STG_Categories to get brand and category names).

2. Lookup Checks whether each incoming product already exists in **Dim_Product** by matching on product_id. This prevents duplicate records from being inserted into the dimension table.

- **Match Output** → record already exists → **ignored**, no action taken.
- **No Match Output** → record is new → passed to the next step.

3. OLE DB Destination Receives only the **new records** (No Match Output) and inserts them into the **Dim_Product** table in the Data Warehouse.

Dim Store



This **Data Flow Task** handles the loading of **Dim_Store** from the Staging layer into the Data Warehouse — identical in structure to **Dim_Product**.

1. OLE DB Source Reads store data from **STG_Stores**, including store name, city, state, and zip code.

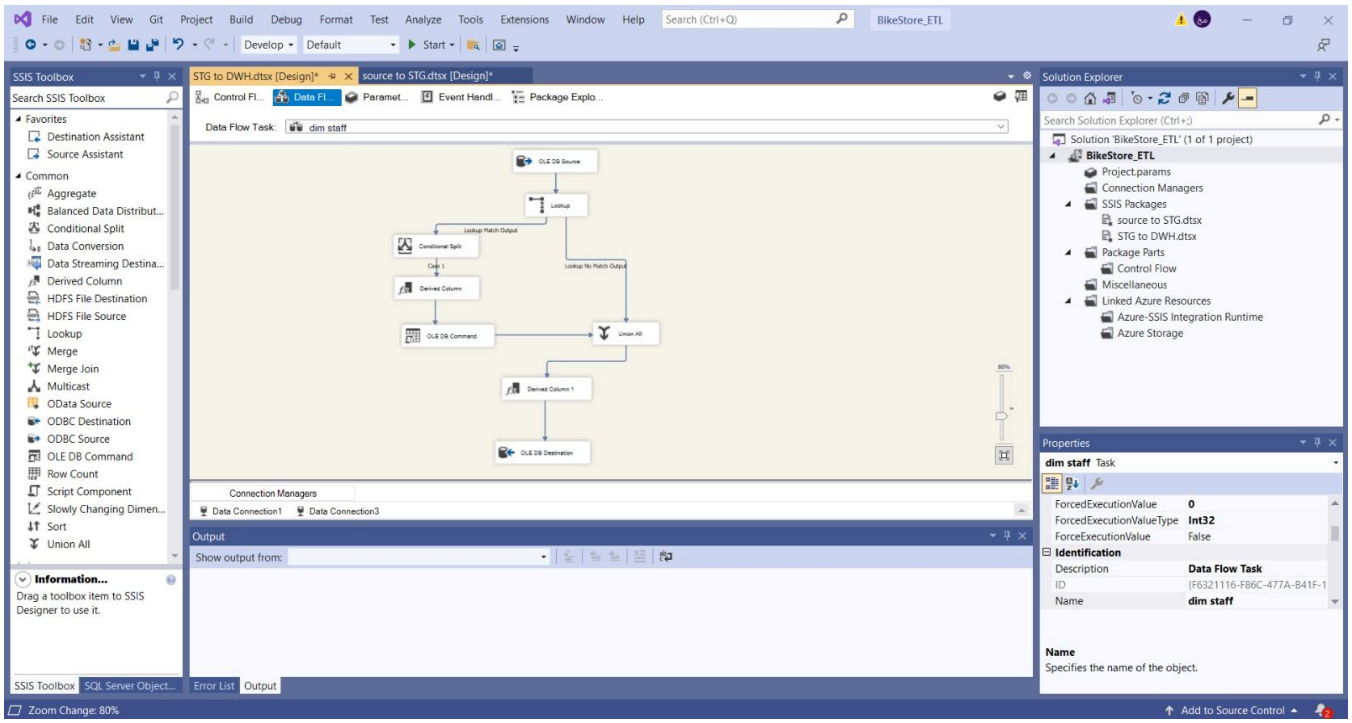
2. Lookup Checks whether each store already exists in **Dim_Store** by matching on store_id.

- **Match Output** → store already exists → **ignored**, no action taken
- **No Match Output** → new store → passed to **OLE DB Destination**

3. OLE DB Destination Inserts only the **new records** into **Dim_Store** in the Data Warehouse.

This implements a simple **SCD Type 1 (insert-only)** strategy — only new stores that don't already exist in the dimension are loaded, with no history tracking required.

Dim Staff (SCD Type 2)



This **Data Flow Task** handles the loading of **Dim_Staff** and implements **Slowly Changing Dimension Type 2 (SCD Type 2)** to preserve the full history of staff changes.

The flow consists of the following components:

1. OLE DB Source Reads staff data from **STG_Staffs**, concatenating first_name and last_name into a single full_name field.

2. Lookup Checks whether each staff member already exists in **Dim_Staff** by matching on staff_id where is_current = 1 (active records only).

- **Match Output** → staff exists → sent to **Conditional Split**
- **No Match Output** → new staff → sent directly to **Union All** for insertion

3. Conditional Split Compares the incoming data with the existing record to detect any changes:

- **Case 1 (Changed)** → full_name or store_id has changed → sent to **OLE DB Command**
- **Case 2 (Unchanged)** → no changes detected → discarded, no action taken

4. OLE DB Command Updates the **old record** in Dim_Staff by setting is_current = 0 and valid_to = today, effectively closing the historical record.

5. Derived Column Prepares the **new record** by setting valid_from = today, valid_to = NULL, and is_current = 1.

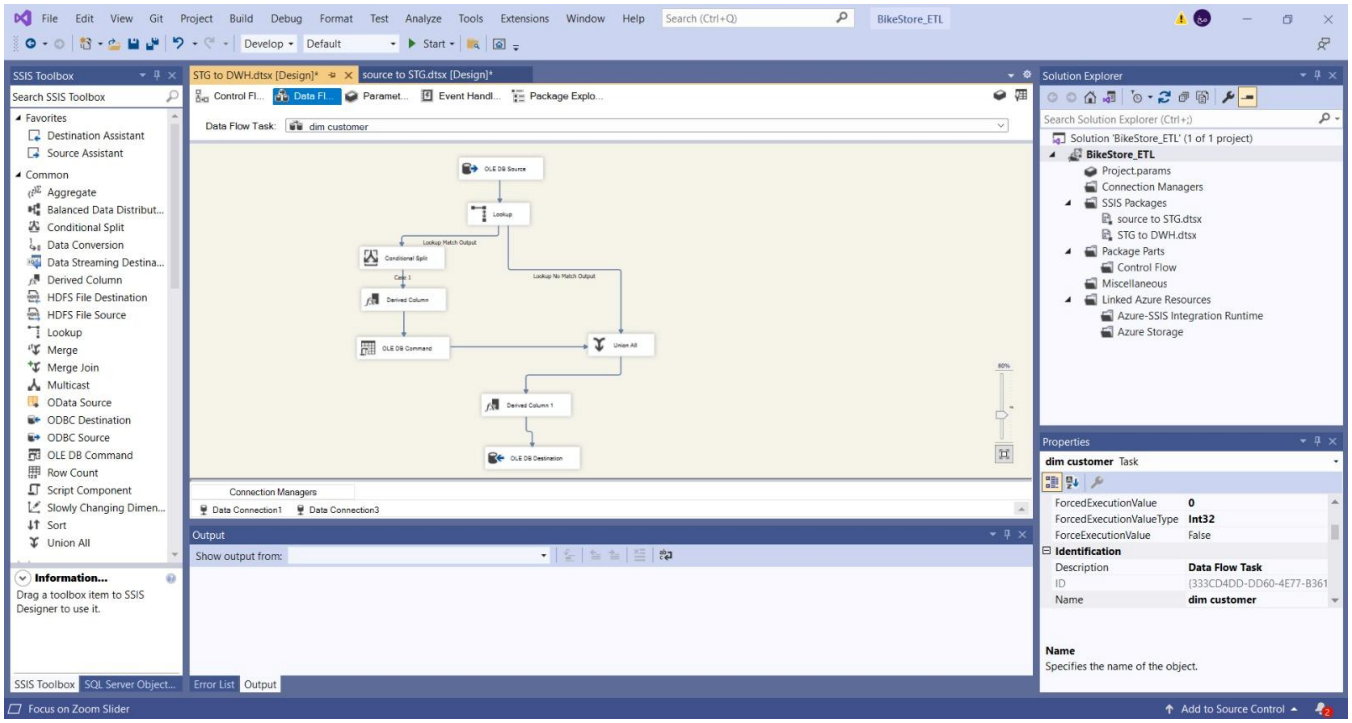
6. Union All Merges both new staff members and changed staff records into a single stream.

7. Derived Column 1 Applies any final column transformations before loading.

8. OLE DB Destination Inserts the final records into **Dim_Staff** in the Data Warehouse.

This SCD Type 2 implementation ensures that every time a staff member's full_name or store assignment changes, the old record is closed and a new row is inserted, preserving the complete history of changes over time.

Dim Customer (SCD Type 2)



This **Data Flow Task** handles the loading of **Dim_Customer** and implements **Slowly Changing Dimension Type 2 (SCD Type 2)** to preserve the full history of customer changes - identical in structure to Dim_Staff.

1. OLE DB Source Reads customer data from **STG_Customers**, combining first_name and last_name into full_name.

2. Lookup Checks whether each customer already exists in **Dim_Customer** by matching on customer_id where is_current = 1.

- **Match Output** → customer exists → sent to **Conditional Split**
- **No Match Output** → new customer → sent directly to **Union All** for insertion

3. Conditional Split Detects any changes in the existing record:

- **Case 1 (Changed)** → full_name or city has changed → sent to **OLE DB Command**
- **Case 2 (Unchanged)** → no changes → discarded

4. OLE DB Command Updates the **old record** by setting is_current = 0 and valid_to = today, closing the historical record.

5. Derived Column Prepares the **new record** with valid_from = today, valid_to = NULL, and is_current = 1.

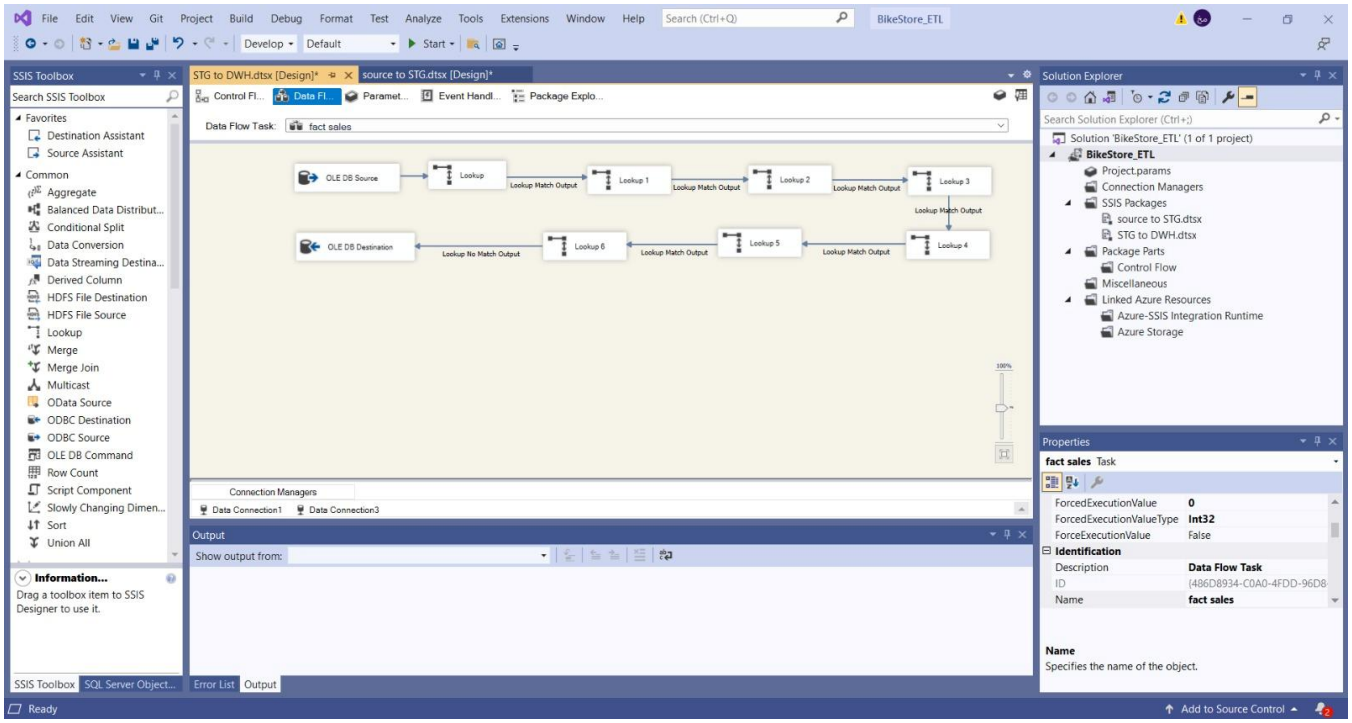
6. Union All Merges new customers and changed customer records into one stream.

7. Derived Column 1 Applies final column transformations before loading.

8. OLE DB Destination Inserts the final records into **Dim_Customer** in the Data Warehouse.

Same SCD Type 2 logic as Dim_Staff — every time a customer's full_name or city changes, the old record is **closed** and a **new row is inserted**, keeping the full history of customer data over time.

Fact Sales



This **Data Flow Task** loads data into **Fact_Sales** by joining staging tables and resolving all surrogate keys through a chain of **6 Lookups**.

1. OLE DB Source Reads data from **STG_Order_Items** joined with **STG_Orders** to get the full sales transaction details including product, customer, store, staff, and order information.

2. Lookup 1 – Date Key Resolves the order_date into the surrogate key date_key from **Dim_Date**.

3. Lookup 2 – Customer Key Matches customer_id to get the surrogate key customer_key from **Dim_Customer** where is_current = 1.

4. Lookup 3 – Product Key Matches product_id to get the surrogate key product_key from **Dim_Product**.

5. Lookup 4 – Store Key Matches store_id to get the surrogate key store_key from **Dim_Store**.

6. Lookup 5 – Staff Key Matches staff_id to get the surrogate key staff_key from **Dim_Staff** where is_current = 1.

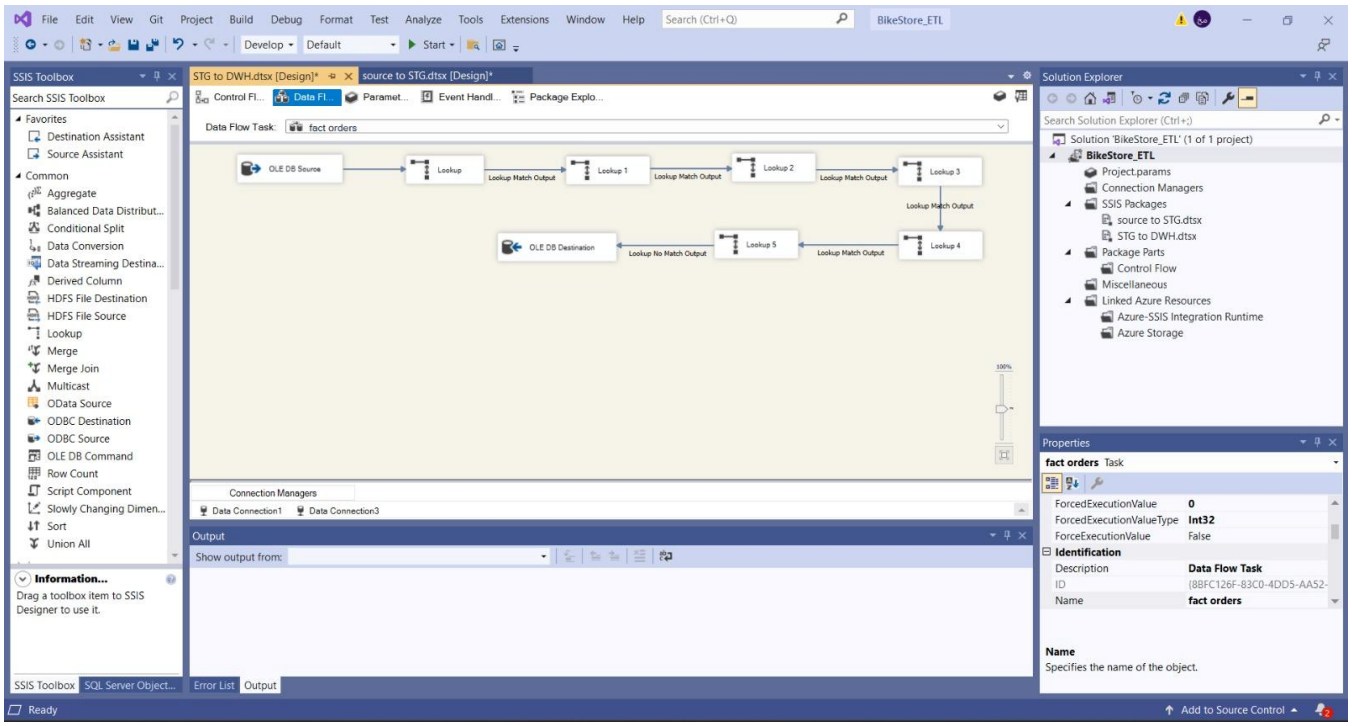
7. Lookup 6 – Duplicate Check The final Lookup checks whether this record **already exists** in **Fact_Sales** to prevent duplicate inserts.

- **Match Output** → record already exists → **discarded**
- **No Match Output** → new record → sent to **OLE DB Destination**

8. OLE DB Destination Inserts only the **new records** into **Fact_Sales** with all surrogate keys resolved and measures calculated (quantity, list_price, discount, total_amount).

The chain of lookups ensures full **referential integrity** between the Fact Table and all its related Dimensions before any data is loaded into the Data Warehouse.

Fact Orders



This **Data Flow Task** loads data into **Fact_Orders**, which represents each order summarized at the order level. It uses **5 Lookups** to resolve all surrogate keys.

1. OLE DB Source Reads data from **STG_Orders** joined with **STG_Order_Items**, aggregating the line items using GROUP BY order_id to calculate total_items and total_amount per order.

2. Lookup – Date Key Resolves order_date into the surrogate key date_key from **Dim_Date**.

3. Lookup 1 – Customer Key Matches customer_id to get customer_key from **Dim_Customer** where is_current = 1.

4. Lookup 2 – Store Key Matches store_id to get store_key from **Dim_Store**.

5. Lookup 3 – Staff Key Matches staff_id to get staff_key from **Dim_Staff** where is_current = 1

6. Lookup 4 – Order Status Key Matches the order status value to get order_status_key from **Dim_Order_Status** (Junk Dimension).

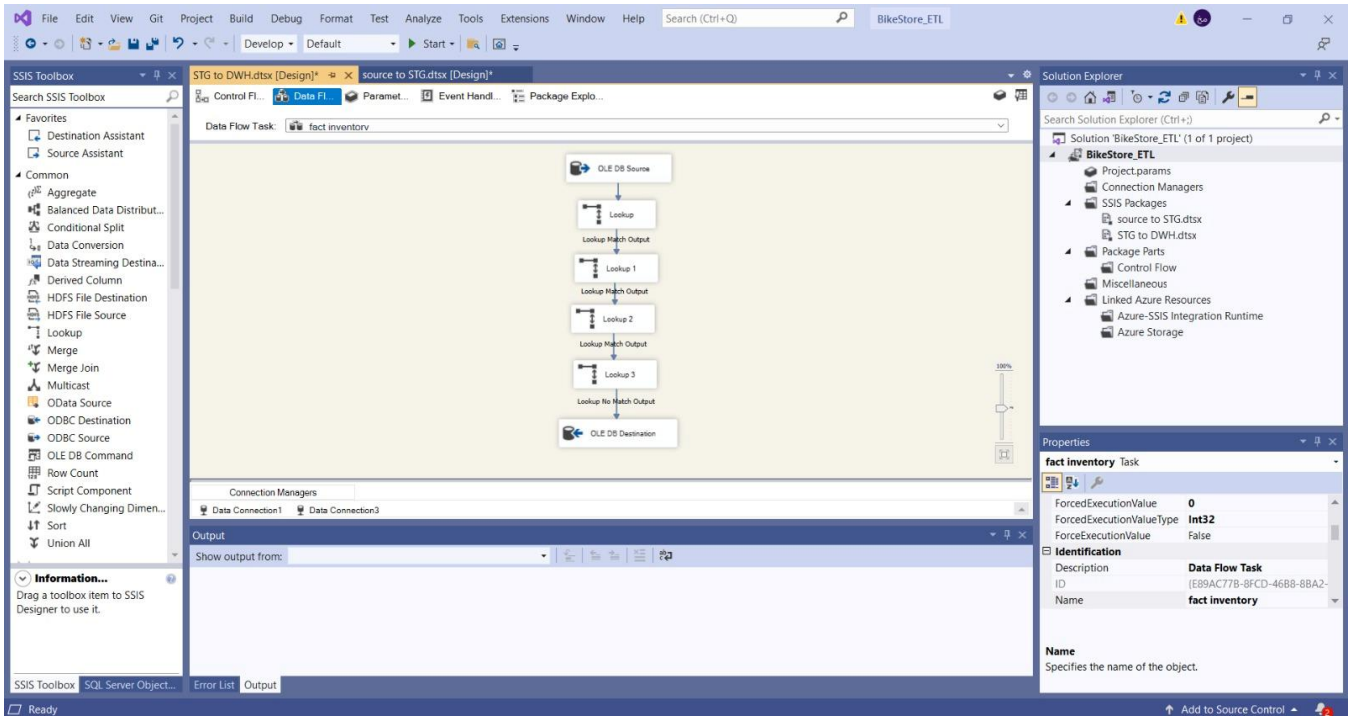
7. Lookup 5 – Duplicate Check Checks whether this order already exists in **Fact_Orders** to prevent duplicate inserts.

- **Match Output** → already exists → **discarded**
- **No Match Output** → new record → sent to **OLE DB Destination**

8. OLE DB Destination Inserts the final aggregated order records into **Fact_Orders** with all surrogate keys resolved and measures loaded (total_items, total_amount).

Unlike Fact_Sales which works at the **line-item level**, Fact_Orders works at the **order level** — each row represents one complete order with its total revenue and item count summarized.

Fact Inventory



This **Data Flow Task** loads data into **Fact_Inventory**, which represents a **periodic snapshot** of stock levels per product per store. It uses **4 Lookups** executed vertically in sequence.

1. OLE DB Source Reads current stock data from **STG_Stocks**, using GETDATE() as the snapshot_date to capture the exact point in time the ETL was run.

2. Lookup – Date Key Resolves the snapshot_date into the surrogate key date_key from **Dim_Date**.

3. Lookup 1 – Product Key Matches product_id to get product_key from **Dim_Product**.

4. Lookup 2 – Store Key Matches store_id to get store_key from **Dim_Store**.

5. Lookup 3 – Duplicate Check The final Lookup checks whether a snapshot for this **product + store + date** combination already exists in **Fact_Inventory** to prevent loading the same snapshot twice.

- **Match Output** → already exists → **discarded**
- **No Match Output** → new snapshot → sent to **OLE DB Destination**

6. OLE DB Destination Inserts the new inventory snapshot records into **Fact_Inventory** with all surrogate keys resolved and the quantity measure loaded.

Unlike Fact_Sales and Fact_Orders which are **transactional**, Fact_Inventory is a **Periodic Snapshot** — meaning every time the ETL runs, it captures the current stock level at that moment in time, building a historical picture of inventory changes over time.

4. Analytical Queries

The following queries were executed on each Fact Table to extract business insights from the Data Warehouse.

Fact_Sales Queries

Fact_Sales represents individual product line items within orders. Each row = one product sold in one order.

Query 1 — Total Sales Revenue and Quantity per Product

Insight: Identifies best-selling products by revenue and quantity. Helps inventory and purchasing decisions.

```sql
SELECT
    p.product_name,
    p.brand_name,
    p.category_name,
    SUM(f.quantity)      AS total_quantity,
    SUM(f.total_amount) AS total_revenue
FROM Fact_Sales f
JOIN Dim_Product p ON f.product_sk = p.product_sk
GROUP BY p.product_name, p.brand_name, p.category_name
ORDER BY total_revenue DESC;
```

150 %

Results | Messages

	product_name	brand_name	category_name	total_quantity	total_revenue
1	Trek Slash 8 27.5 - 2016	Trek	Mountain Bikes	154	555558.46
2	Trek Conduit+ - 2016	Trek	Electric Bikes	145	389248.55
3	Trek Fuel EX 8 29 - 2016	Trek	Mountain Bikes	143	368472.57
4	Surly Straggler 650b - 2016	Surly	Cyclocross Bicycles	151	226765.39
5	Trek Domane SLR 6 Disc - 2017	Trek	Road Bikes	43	211584.57
6	Surly Straggler - 2016	Surly	Cyclocross Bicycles	147	203507.62
7	Trek Remedy 29 Carbon Frameset - 2016	Trek	Mountain Bikes	125	203380.75
8	Trek Powerfly 8 FS Plus - 2017	Trek	Electric Bikes	41	188249.59
9	Trek Madone 9.2 - 2017	Trek	Road Bikes	39	175899.61
10	Trek Silque SLR 8 Women's - 2017	Trek	Road Bikes	29	174524.71
11	Trek Silque SLR 7 Women's - 2017	Trek	Road Bikes	30	163079.70
12	Trek Fuel EX 9.8 27.5 Plus - 2017	Trek	Mountain Bikes	33	159052.67
13	Heller Shagamaw Frame - 2016	Heller	Mountain Bikes	129	151160.74
14	Trek Fuel EX 9.8 29 - 2017	Trek	Mountain Bikes	33	147449.67
15	Trek Boone 7 - 2017	Trek	Cyclocross Bicycles	42	127609.58
16	Trek Domane SL 6 - 2017	Trek	Road Bikes	37	113644.63

Result: Trek Slash 8 27.5 - 2016 (Mountain Bikes) generated the highest revenue of $555,558 with 154 units sold.

Query 2 — Total Revenue per Store per Year

Insight: Compares store performance across years. Identifies growth trends and underperforming locations.

```sql
SELECT
    s.store_name,
    d.year,
    SUM(f.total_amount) AS total_revenue
FROM Fact_Sales f
JOIN Dim_Store  s ON f.store_sk  = s.store_sk
JOIN Dim_Date   d ON f.date_sk   = d.date_sk
GROUP BY s.store_name, d.year
ORDER BY d.year, total_revenue DESC;
```

150 %

Results | Messages

	store_name	year	total_revenue
1	Baldwin Bikes	2016	1590597.24
2	Santa Cruz Bikes	2016	565493.02
3	Rowlett Bikes	2016	271285.91
4	Baldwin Bikes	2017	2479686.19
5	Santa Cruz Bikes	2017	562907.63
6	Rowlett Bikes	2017	404611.24
7	Baldwin Bikes	2018	1145463.21
8	Santa Cruz Bikes	2018	477420.91
9	Rowlett Bikes	2018	191644.36

Result: Baldwin Bikes consistently ranks first across all years, generating $2.4M in 2017.

Query 3 — Top 10 Customers by Total Spending

Insight: Identifies the most valuable customers for loyalty programs and targeted marketing.

```sql
SELECT TOP 10
    c.full_name,
    c.city,
    c.state,
    SUM(f.total_amount) AS total_spent
FROM Fact_Sales f
JOIN Dim_Customer c ON f.customer_sk = c.customer_sk
WHERE c.is_current = 1
GROUP BY c.full_name, c.city, c.state
ORDER BY total_spent DESC;
```

150 %

Results | Messages

	full_name	city	state	total_spent
1	Sharyn Hopkins	Baldwinsville	NY	34807.93
2	Pamelia Newman	Monroe	NY	33634.24
3	Abby Gamble	Amityville	NY	32802.99
4	Lyndsey Bean	Fairport	NY	32675.06
5	Emmitt Sanchez	New York	NY	31925.87
6	Melanie Hayes	Liverpool	NY	31913.68
7	Debra Burks	Orchard Park	NY	27888.17
8	Elinore Aguilar	San Angelo	TX	25636.43
9	Corrina Sawyer	Troy	NY	25612.69
10	Shena Carter	Howard Beach	NY	24890.62

Result: Sharyn Hopkins from Baldwinsville, NY leads with $34,807 in total spending.

Query 4 — Total Sales per Staff Member

Insight: Measures staff productivity and sales performance for HR and commission decisions.

```sql
SELECT
    st.full_name    AS staff_name,
    COUNT(DISTINCT f.order_id) AS total_orders,
    SUM(f.total_amount)        AS total_revenue
FROM Fact_Sales f
JOIN Dim_Staff st ON f.staff_sk = st.staff_sk
WHERE st.is_current = 1
GROUP BY st.full_name
ORDER BY total_revenue DESC;
```

150 %

Results | Messages

	staff_name	total_orders	total_revenue
1	Marcelene Boyer	553	2624118.25
2	Venita Daniel	540	2591628.39
3	Genna Serrano	184	853286.55
4	Mireya Copeland	164	752535.01
5	Kali Vargas	88	463917.91
6	Layla Terrell	86	403623.60

Result: Marcelene Boyer leads with 553 orders totaling $2.6M in revenue.

Fact_Orders Queries

Fact_Orders represents complete customer orders. Each row = one order summarized at the order level.

Query 1 — Order Distribution by Status

Insight: Shows the proportion of completed, pending, processing, and rejected orders for operational monitoring.

```sql
SELECT
    os.status_label,
    COUNT(f.order_id)   AS total_orders,
    SUM(f.total_amount) AS total_revenue
FROM Fact_Orders f
JOIN Dim_Order_Status os ON f.order_status_sk = os.order_status_sk
GROUP BY os.status_label
ORDER BY total_orders DESC;
```

150 %

Results | Messages

	status_label	total_orders	total_revenue
1	Completed	1445	6662609.80
2	Processing	63	429182.05
3	Pending	62	388739.29
4	Rejected	45	208579.30

Result: 1,445 orders are Completed (89%), 63 Processing, 62 Pending, 45 Rejected.

Query 2 — Average Order Value per Store

Insight: Compares average transaction size across stores to assess pricing and customer behavior.

```sql
SELECT
    s.store_name,
    COUNT(f.order_id)        AS total_orders,
    AVG(f.total_amount)      AS avg_order_value,
    SUM(f.total_amount)      AS total_revenue
FROM Fact_Orders f
JOIN Dim_Store s ON f.store_sk = s.store_sk
GROUP BY s.store_name
ORDER BY total_revenue DESC;
```

150 %

Results | Messages

	store_name	total_orders	avg_order_value	total_revenue
1	Baldwin Bikes	1093	4771.955288	5215747.13
2	Santa Cruz Bikes	348	4614.430172	1605821.70
3	Rowlett Bikes	174	4985.871321	867541.61

Result: Baldwin Bikes handles 1,093 orders with an average value of $4,772 per order.

Query 3 — Monthly Order Volume and Revenue

Insight: Reveals seasonal trends and peak sales months to guide promotions and stock planning.

```sql
SELECT
    d.year,
    d.month_name,
    COUNT(f.order_id)   AS total_orders,
    SUM(f.total_amount) AS total_revenue
FROM Fact_Orders f
JOIN Dim_Date d ON f.date_sk = d.date_sk
GROUP BY d.year, d.month_name, d.month
ORDER BY d.year, d.month;
```

150 %

Results | Messages

	year	month_name	total_orders	total_revenue
1	2016	January	50	215146.24
2	2016	February	49	156112.15
3	2016	March	55	180600.16
4	2016	April	43	167143.91
5	2016	May	51	205269.83
6	2016	June	45	210561.96
7	2016	July	50	199556.62
8	2016	August	63	225657.16
9	2016	September	67	273091.40
10	2016	October	64	212077.87
11	2016	November	43	182329.26
12	2016	December	55	199829.79
13	2017	January	50	285616.27
14	2017	February	57	312923.52
15	2017	March	67	308911.66
16	2017	April	57	227290.68
17	2017	May	57	268233.02
18	2017	June	63	378865.38
19	2017	July	52	229995.17

Result: September 2016 shows the highest single-month volume with 67 orders.

Query 4 — Top 10 Most Active Customers

Insight: Identifies customers who order most frequently — key for retention and VIP programs.

```sql
SELECT TOP 10
    c.full_name,
    c.city,
    COUNT(f.order_id)   AS total_orders,
    SUM(f.total_amount) AS total_spent
FROM Fact_Orders f
JOIN Dim_Customer c ON f.customer_sk = c.customer_sk
WHERE c.is_current = 1
GROUP BY c.full_name, c.city
ORDER BY total_orders DESC;
```

150 %

Results | Messages

	full_name	city	total_orders	total_spent
1	Corene Wall	Atwater	3	14199.94
2	Latasha Hays	Buffalo	3	7147.16
3	Kasha Todd	Campbell	3	19329.07
4	Georgetta Hardin	Canandaigua	3	14877.27
5	Bobbie Foster	Desoto	3	8778.12
6	Monika Berg	Encino	3	8902.50
7	Ronna Butler	Encino	3	17240.55
8	Merlene Vinson	Euless	3	12793.62
9	Lyndsey Bean	Fairport	3	32675.06
10	Garry Espinoza	Forney	3	15766.69

Result: Multiple customers have placed 3 orders each, with Lyndsey Bean from Fairport spending the most ($32,675).

Fact_Inventory Queries

Fact_Inventory represents a periodic snapshot of stock levels. Each row = current quantity of one product in one store.

Query 1 — Total Stock per Store

Insight: Shows overall inventory distribution across stores to identify over-stocked or under-stocked locations.

```sql
SELECT
    s.store_name,
    SUM(f.quantity) AS total_stock
FROM Fact_Inventory f
JOIN Dim_Store s ON f.store_sk = s.store_sk
GROUP BY s.store_name
ORDER BY total_stock DESC;
```

150 %

Results | Messages

	store_name	total_stock
1	Rowlett Bikes	4620
2	Santa Cruz Bikes	4532
3	Baldwin Bikes	4359

Result: Rowlett Bikes holds the most stock (4,620 units), followed by Santa Cruz Bikes (4,532).

Query 2 — Top 10 Least Stocked Products

Insight: Flags products at risk of stockout for replenishment prioritization.

```sql
SELECT TOP 10
    p.product_name,
    p.brand_name,
    p.category_name,
    SUM(f.quantity) AS total_stock
FROM Fact_Inventory f
JOIN Dim_Product p ON f.product_sk = p.product_sk
GROUP BY p.product_name, p.brand_name, p.category_name
ORDER BY total_stock ASC;
```

150 %

Results | Messages

	product_name	brand_name	category_name	total_stock
1	Trek Domane SLR Frameset - 2018	Trek	Road Bikes	5
2	"Electra Superbolt 1 20"" - 2018"	Electra	Children Bicycles	9
3	Electra Cruiser 1 Ladies' - 2018	Electra	Cruisers Bicycles	10
4	Electra Amsterdam Original 3i - 2015/2017	Electra	Cruisers Bicycles	11
5	Sun Bicycles Streamway 7 - 2017	Sun Bicycles	Comfort Bicycles	11
6	Electra Amsterdam Original 3i Ladies' - 2017	Electra	Cruisers Bicycles	12
7	Electra Koa 3i Ladies' - 2018	Electra	Cruisers Bicycles	12
8	Electra Townie Commute Go! - 2018	Electra	Cruisers Bicycles	12
9	Electra Townie Original 1 - 2018	Electra	Comfort Bicycles	12
10	Trek Silque SLR 8 Women's - 2017	Trek	Road Bikes	12

Result: Trek Domane SLR Frameset - 2018 has the lowest stock with only 5 total units across all stores.

Query 3 — Stock Distribution per Category

Insight: Shows how inventory is allocated across product categories for portfolio planning.

```sql
SELECT
    p.category_name,
    COUNT(DISTINCT p.product_sk) AS total_products,
    SUM(f.quantity)              AS total_stock
FROM Fact_Inventory f
JOIN Dim_Product p ON f.product_sk = p.product_sk
GROUP BY p.category_name
ORDER BY total_stock DESC;
```

150 %

Results | Messages

	category_name	total_products	total_stock
1	Cruisers Bicycles	78	3378
2	Mountain Bikes	60	2654
3	Children Bicycles	59	2608
4	Road Bikes	54	2091
5	Comfort Bicycles	28	1258
6	Electric Bikes	24	1108
7	Cyclocross Bicycles	10	414

Result: Cruisers Bicycles dominate with 78 products and 3,378 units. Cyclocross Bicycles have the lowest stock (414 units).

Query 4 — Zero Stock Products Insight

Identifies products with zero inventory across stores, enabling immediate restocking action to prevent lost sales

```sql
SELECT
    p.product_name,
    p.brand_name,
    s.store_name,
    f.quantity
FROM Fact_Inventory f
JOIN Dim_Product p ON f.product_sk = p.product_sk
JOIN Dim_Store   s ON f.store_sk   = s.store_sk
WHERE f.quantity = 0
ORDER BY p.product_name;
```

150 %

Results | Messages

	category_name	total_products	total_stock
1	Cruisers Bicycles	78	3378
2	Mountain Bikes	60	2654
3	Children Bicycles	59	2608
4	Road Bikes	54	2091
5	Comfort Bicycles	28	1258
6	Electric Bikes	24	1108
7	Cyclocross Bicycles	10	414

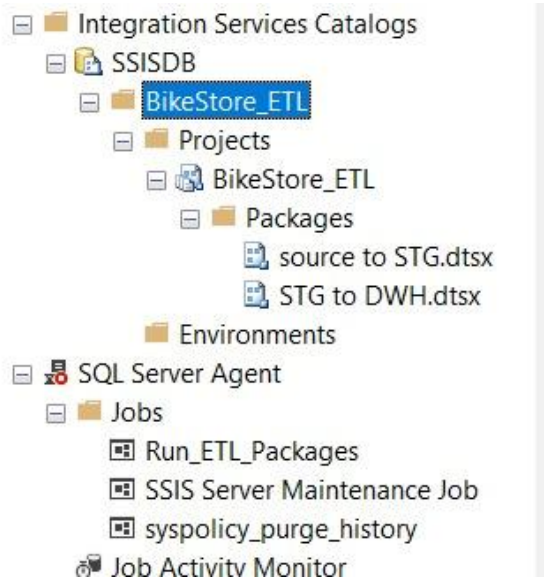
Result :Products with zero quantity are retrieved per store location, sorted alphabetically — directly highlighting which items need urgent replenishment and in which specific stores.

---

5. Deployment and Scheduling

SSIS Catalog Deployment

This screenshot shows the **deployment structure** of the BikeStore ETL solution in SQL Server Management Studio (SSMS).



Integration Services Catalogs – SSISDB

Both SSIS packages were deployed to the **SSISDB Catalog** under the project **BikeStore_ETL**:

- **source to STG.dtsx** → Extracts data from the OLTP source and loads it into the Staging area
- **STG to DWH.dtsx** → Transforms and loads data from Staging into the Data Warehouse

The catalog provides centralized **execution monitoring, logging, and management** for both packages.

SQL Server Agent – Jobs

A **SQL Server Agent Job** named **Run_ETL_Packages** was created to automate the ETL process, executing both packages sequentially:

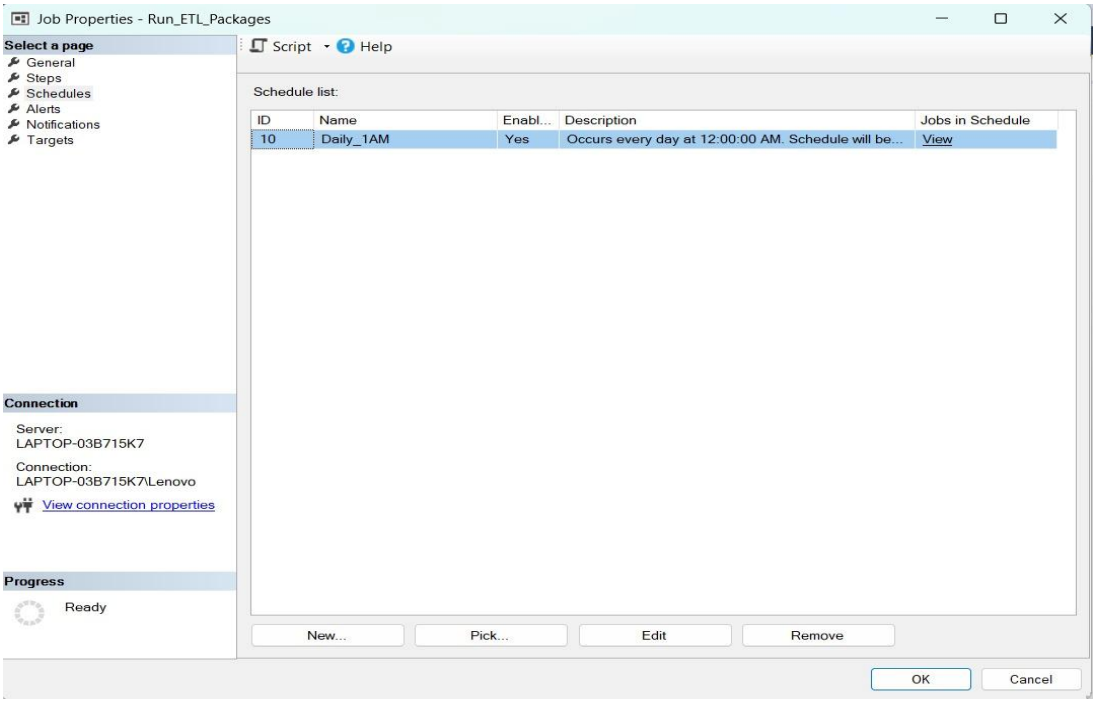
1. **Step 1** → runs source to STG.dtsx
2. **Step 2** → runs STG to DWH.dtsx

The job is scheduled to run **daily at midnight**, ensuring the Data Warehouse is refreshed automatically with the latest data from the OLTP source every day.

The other two jobs (SSIS Server Maintenance Job and syspolicy_purge_history) are default SQL Server system jobs unrelated to the ETL pipeline.

SQL Server Agent Job - Schedule Configuration

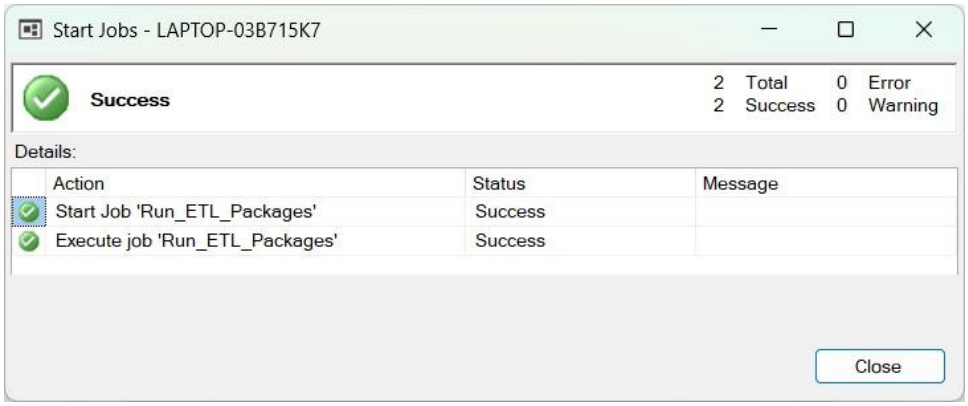
This screenshot shows the **Schedule settings** of the **Run_ETL_Packages** job in SQL Server Agent.



Property	Value
Job Name	Run_ETL_Packages
Schedule Name	Daily_1AM
Frequency	Daily
Start Time	12:00:00 AM
Status	Enabled

The job was successfully executed manually on 5/8/2026 at 8:29 PM with both steps completing successfully. The automated schedule runs daily at midnight to refresh the Data Warehouse with the latest data from the OLTP source.

SQL Server Agent Job – Execution Result:



Execution Summary:

Property	Value
Total Steps	2
Success	2
Errors	0
Warnings	0

Details:

Action	Status
Start Job 'Run_ETL_Packages'	Success
Execute Job 'Run_ETL_Packages'	Success

Both steps completed **successfully with no errors or warnings**, confirming that the full ETL pipeline ran end-to-end — from extracting data out of the OLTP source, loading it into the Staging area,

transforming it, and finally loading it into the Data Warehouse.

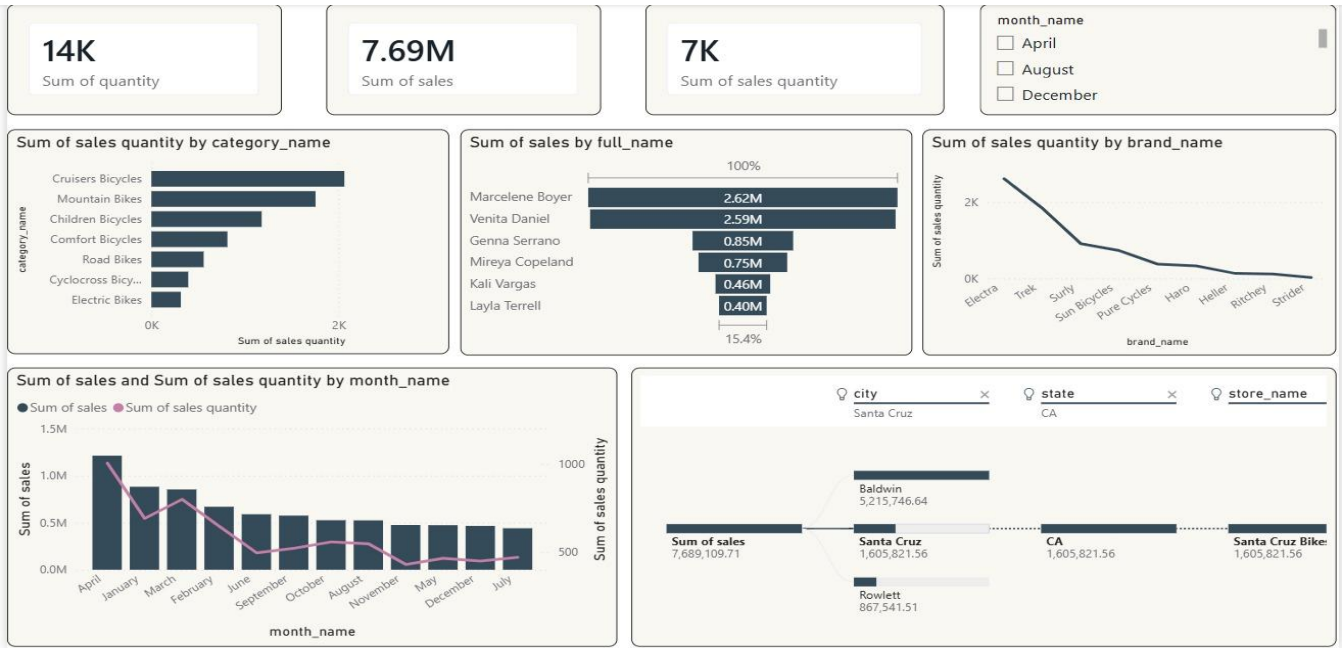
---

6. Power BI Dashboard – BikeStore Sales Analysis (Bonus)

This **Power BI Dashboard** was built on top of the Data Warehouse to provide visual business insights across sales, quantity, and performance metrics.

KPI Cards (Top Summary)

- **14K** → Total Sum of Quantity sold
- **7.69M** → Total Sum of Sales Revenue
- **7K** → Total Sum of Sales Quantity



Charts & Visuals

1. Sum of Sales Quantity by Category Horizontal bar chart showing **Cruisers Bicycles** as the top-selling category, followed by Mountain Bikes and Children Bicycles. Electric Bikes have the lowest sales quantity.

2. Sum of Sales by Staff (full_name) Bar chart ranking staff by revenue generated. **Marcelene Boyer** leads with **$2.62M**, followed by **Venita Daniel** with **$2.59M**, confirming their dominance in sales performance.

3. Sum of Sales Quantity by Brand Line chart showing **Electra** as the highest-selling brand by quantity, with a sharp drop-off across other brands like Trek, Surly, and Sun Bicycles.

4. Sum of Sales and Quantity by Month Combined bar and line chart showing monthly trends. **January** records the highest revenue (~$1.25M), with a gradual decline toward mid-year months.

5. Geographic Filter Panel An interactive slicer filtered by **City (Santa Cruz)**, **State (CA)**, and **Store Name**, showing:

- **Baldwin** → $5.2M total sales
- **Santa Cruz Bikes** → $1.6M
- **Rowlett** → $867K

➔ The dashboard also includes a **month filter slicer** (April, August, December) for dynamic time-based analysis across all visuals.

Thank you!